Estructuras de datos

Actividad 2

Presentado A

Jose Ruiz

Presentado Por

Cristian Stiven Guerrero

Andrade

ID 1032796841

Enlace Repositorio Proyecto:
https://github.com/demondcn/EstructuraDeDatosBibliotec

Corporación Universitaria Iberoamericana De Colombia

Abril 2026

# 1. introducción

En la actualidad, el desarrollo de software orientado a la administración de información requiere no solo conocimientos de programación, sino también una adecuada selección de estructuras de datos que permitan organizar, almacenar y procesar la información de manera eficiente. En este contexto, las estructuras de datos lineales constituyen una base fundamental dentro de la formación en ciencias de la computación y desarrollo de software, ya que ofrecen soluciones claras para representar secuencias de elementos y modelar problemas del mundo real.

El presente trabajo tiene como propósito exponer el proceso de diseño, desarrollo y validación del prototipo funcional IberoBibliopyStructureDates, un sistema de gestión de biblioteca elaborado en Python con interfaz gráfica en Tkinter. Este prototipo fue concebido como una aplicación académica orientada a demostrar la implementación práctica de estructuras de datos lineales, específicamente listas, colas, pilas y arreglos, dentro de un caso de uso real: la administración de libros, usuarios, préstamos, devoluciones y reservas en una biblioteca.

El proyecto responde al resultado de aprendizaje centrado en comunicar adecuadamente los parámetros de los datos a almacenar, seleccionando tipos de datos y estructuras congruentes con los requisitos del sistema. Desde esta perspectiva, no se trata únicamente de construir una interfaz funcional, sino de justificar técnicamente por qué cada estructura fue elegida, cómo se organiza la información y de qué manera dicha organización influye en el comportamiento del sistema. En consecuencia, el trabajo integra la dimensión conceptual, procedimental y práctica del curso, articulando teoría y desarrollo de software en una experiencia aplicada.

Asimismo, el prototipo busca evidenciar que las estructuras de datos lineales no son conceptos aislados ni exclusivamente teóricos, sino herramientas esenciales para resolver necesidades concretas. En una biblioteca, por ejemplo, resulta necesario almacenar colecciones de libros y usuarios, gestionar turnos de espera cuando no existen ejemplares disponibles, registrar acciones recientes y mantener criterios de organización consistentes. Cada una de estas necesidades puede asociarse con una estructura de datos específica, lo cual permite construir un sistema más claro, mantenible y coherente.

Este documento presenta, en primer lugar, el desarrollo temático del proyecto y el sustento conceptual que lo acompaña. Posteriormente, explica el funcionamiento del prototipo funcional, la relación entre los conceptos estudiados y su aplicación práctica, las pruebas realizadas para validar la lógica del sistema y, finalmente, las conclusiones derivadas del proceso de implementación. De esta manera, se ofrece una visión integral del trabajo realizado, destacando tanto el valor académico del ejercicio como su utilidad para comprender el papel de las estructuras de datos en el desarrollo de soluciones informáticas.

# 2. Desarrollo del Tema

Las estructuras de datos lineales constituyen uno de los pilares fundamentales en la formación de cualquier estudiante

de programación, ya que permiten organizar la información siguiendo un orden secuencial y facilitan la implementación de operaciones básicas como inserción, eliminación, búsqueda y recorrido. Su estudio no solo tiene relevancia teórica, sino también práctica, puesto que numerosos sistemas informáticos cotidianos dependen de este tipo de estructuras para operar con eficiencia y claridad.

Dentro del contexto del curso de Estructura de Datos, el proyecto de sistema de gestión de biblioteca representa una oportunidad pertinente para aplicar estos conocimientos en una situación real. Una biblioteca requiere administrar información heterogénea: libros, usuarios, préstamos, devoluciones, reservas e historial de acciones. Aunque a primera vista estas tareas pueden parecer únicamente administrativas, en realidad implican decisiones estructurales importantes relacionadas con la forma en que los datos se almacenan, se vinculan y se recuperan.

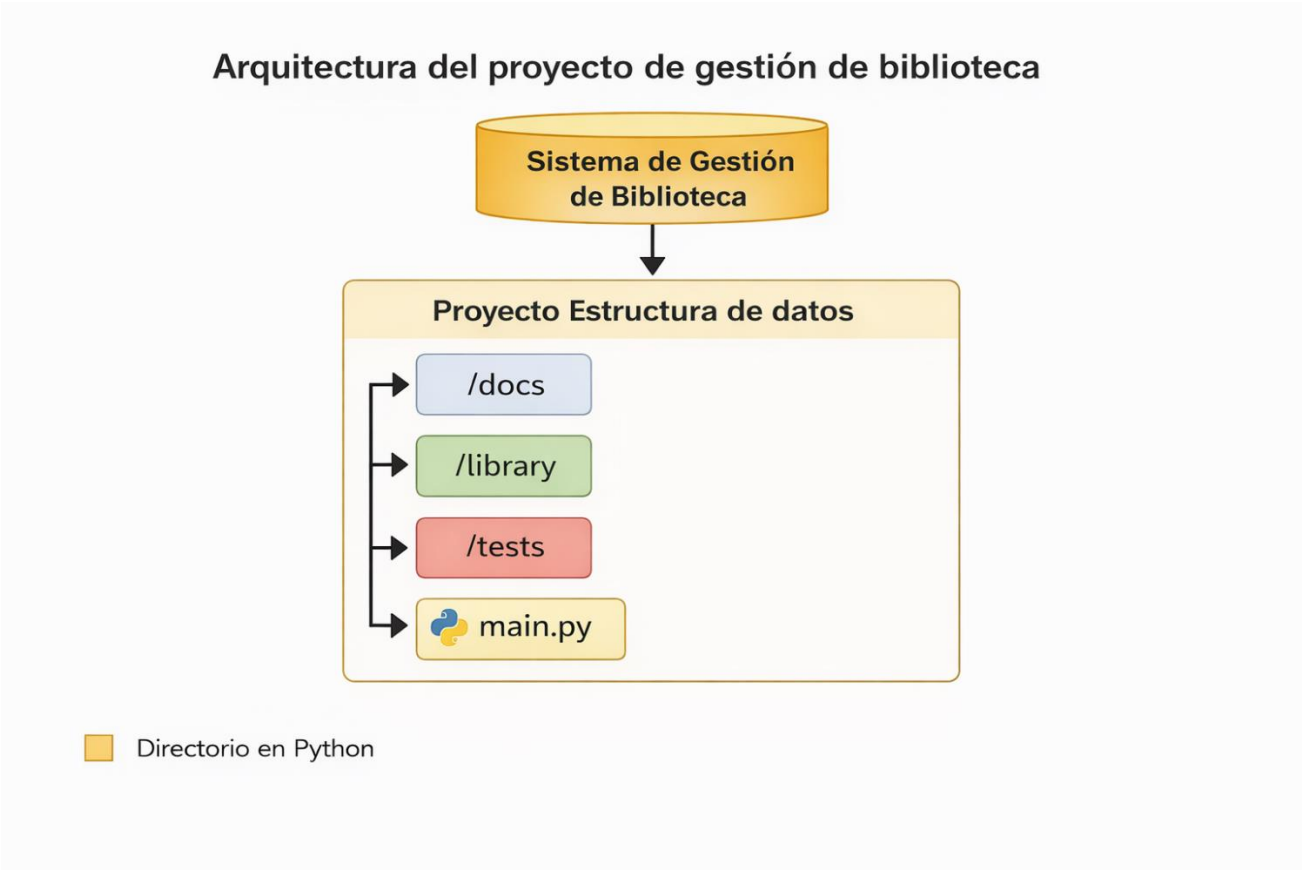


Figura 1. Arquitectura utilizada del proyecto. Fuente: elaboración propia.

Uno de los conceptos centrales que orientó este proyecto fue la parametrización de datos, entendida como la definición clara de qué información debe almacenarse, bajo qué tipo de dato y en qué estructura conviene organizarla según su comportamiento esperado. Esta etapa es clave, porque un sistema correctamente parametrizado facilita tanto la implementación del código como la coherencia funcional del software. En el caso del prototipo desarrollado, se definieron entidades principales como libro, usuario y préstamo, cada una con atributos específicos que responden a necesidades operativas concretas.

Para el registro de los libros se establecieron parámetros como el ISBN, el título, el autor, la categoría, la cantidad total de ejemplares, la cantidad disponible y el tamaño de la cola de reserva. Estos datos no fueron elegidos de manera arbitraria. El ISBN funciona como identificador principal del libro, mientras que el título y el autor permiten búsquedas más accesibles para el usuario. La categoría facilita la organización temática del catálogo, y los campos relacionados con

la disponibilidad hacen posible gestionar el préstamo y la reserva de ejemplares de manera precisa.

En el caso de los usuarios, se definieron como datos esenciales el ID, el nombre y el correo electrónico. Estos atributos permiten identificar a cada usuario de manera individual, realizar búsquedas por distintos criterios y vincular correctamente cada préstamo o reserva con la persona correspondiente. Para los préstamos, por su parte, se establecieron como parámetros el ISBN del libro, el ID del usuario, la fecha del préstamo y el estado de devolución. Esta información hace posible controlar qué libro fue prestado, a quién, en qué momento y si ya fue devuelto o sigue activo.

Una vez definidos los datos a almacenar, el siguiente paso consistió en elegir las estructuras de datos lineales más adecuadas para representarlos. Esta selección fue uno de los aspectos más relevantes del proyecto, ya que permitió traducir la teoría del curso en decisiones concretas de diseño. En lugar de utilizar una única estructura para todo el sistema, se optó por combinar varias estructuras según la naturaleza de cada necesidad funcional.

En primer lugar, se emplearon listas para almacenar libros, usuarios y préstamos. La lista resultó apropiada porque ofrece un manejo dinámico de elementos, permite añadir nuevos registros conforme crece el sistema y facilita el recorrido secuencial para ejecutar búsquedas o mostrar información en pantalla. En un prototipo académico como el desarrollado, la lista representa una alternativa suficientemente flexible y comprensible para gestionar colecciones de datos sin requerir una complejidad adicional innecesaria.

En segundo lugar, se utilizó una cola para la administración de reservas de libros. Esta decisión responde a una lógica claramente funcional: cuando un libro no tiene ejemplares disponibles, varios usuarios pueden quedar en espera, y el orden correcto de atención debe respetar el principio FIFO (First In, First Out), es decir, el primero en reservar debe ser el primero en recibir el libro cuando exista disponibilidad. La cola, por tanto, no solo cumple una función técnica, sino también una función organizativa y ética, al garantizar un tratamiento justo de las solicitudes.

En tercer lugar, se implementó una pila para registrar el historial de acciones recientes del sistema. Esta estructura responde al principio LIFO (Last In, First Out), adecuado para almacenar eventos recientes y consultarlos en orden inverso al que ocurrieron. Su uso resulta pertinente para visualizar las últimas operaciones realizadas, como registros, préstamos o devoluciones, y constituye una manera práctica de aplicar el concepto de pila a una situación concreta dentro del software.

Finalmente, se recurrió a un arreglo de tamaño fijo para representar las categorías base de la biblioteca. Aunque en muchos contextos las listas también podrían utilizarse para esta finalidad, el arreglo permite expresar que se trata de una colección estable y previamente definida, cuyo tamaño no cambia constantemente durante la ejecución del sistema. Esta decisión refuerza la comprensión de que no todas las estructuras deben elegirse por flexibilidad, sino también por adecuación al comportamiento esperado de los datos.

Desde el punto de vista del diseño de software, el proyecto también implicó una reflexión sobre la relación entre lógica interna e interacción con el usuario. Un sistema de biblioteca no puede limitarse a almacenar datos de manera

abstracta; debe permitir que dichos datos sean consultados y administrados mediante una interfaz clara. Por esta razón, el prototipo fue desarrollado en Python y complementado con una interfaz gráfica en Tkinter. Esta combinación permitió construir una aplicación funcional, visualmente ordenada y coherente con el propósito académico del proyecto.

La elección de Tkinter obedeció a varios factores. En primer lugar, se trata de una biblioteca gráfica integrada en Python, lo que facilita su implementación dentro de un entorno de aprendizaje. En segundo lugar, permite desarrollar interfaces suficientemente completas para representar menús, formularios, paneles de información, pestañas y botones de acción. En tercer lugar, ofrece una integración directa con la lógica del sistema, lo que favorece una arquitectura comprensible y apropiada para fines formativos.

En términos visuales, la interfaz fue inspirada en sistemas bibliotecarios reales, particularmente en la lógica de organización observable en plataformas como Koha, aunque adaptada con una identidad propia en español y una paleta de colores negro, amarillo y miel. Esta elección estética no fue superficial. El propósito fue ofrecer una experiencia visual consistente con el contexto de uso, transmitir orden institucional y facilitar la navegación del usuario dentro del sistema. Un diseño claro también contribuye a reducir errores de interacción y mejora la comprensión de las operaciones disponibles.

En síntesis, el desarrollo temático del proyecto demuestra que la construcción de un sistema de gestión de biblioteca no depende únicamente de la capacidad de programar interfaces o formularios, sino de la habilidad para analizar requisitos, parametrizar datos, seleccionar estructuras adecuadas y articular teoría y práctica de forma coherente. El prototipo IberoBibliopyStructureDates, por tanto, constituye una aplicación concreta del estudio de las estructuras de datos lineales y evidencia cómo estos conceptos pueden convertirse en soluciones funcionales para necesidades reales de administración de información.

## 3. Explicación del prototipo funcional

El prototipo funcional desarrollado recibe el nombre de IberoBibliopyStructureDates y fue diseñado como un sistema de gestión de biblioteca orientado a la administración de libros, usuarios, préstamos, devoluciones, reservas e historial de acciones. Su propósito principal es demostrar de manera práctica la correcta utilización de estructuras de datos lineales en Python, integradas dentro de una interfaz gráfica intuitiva construida con Tkinter.

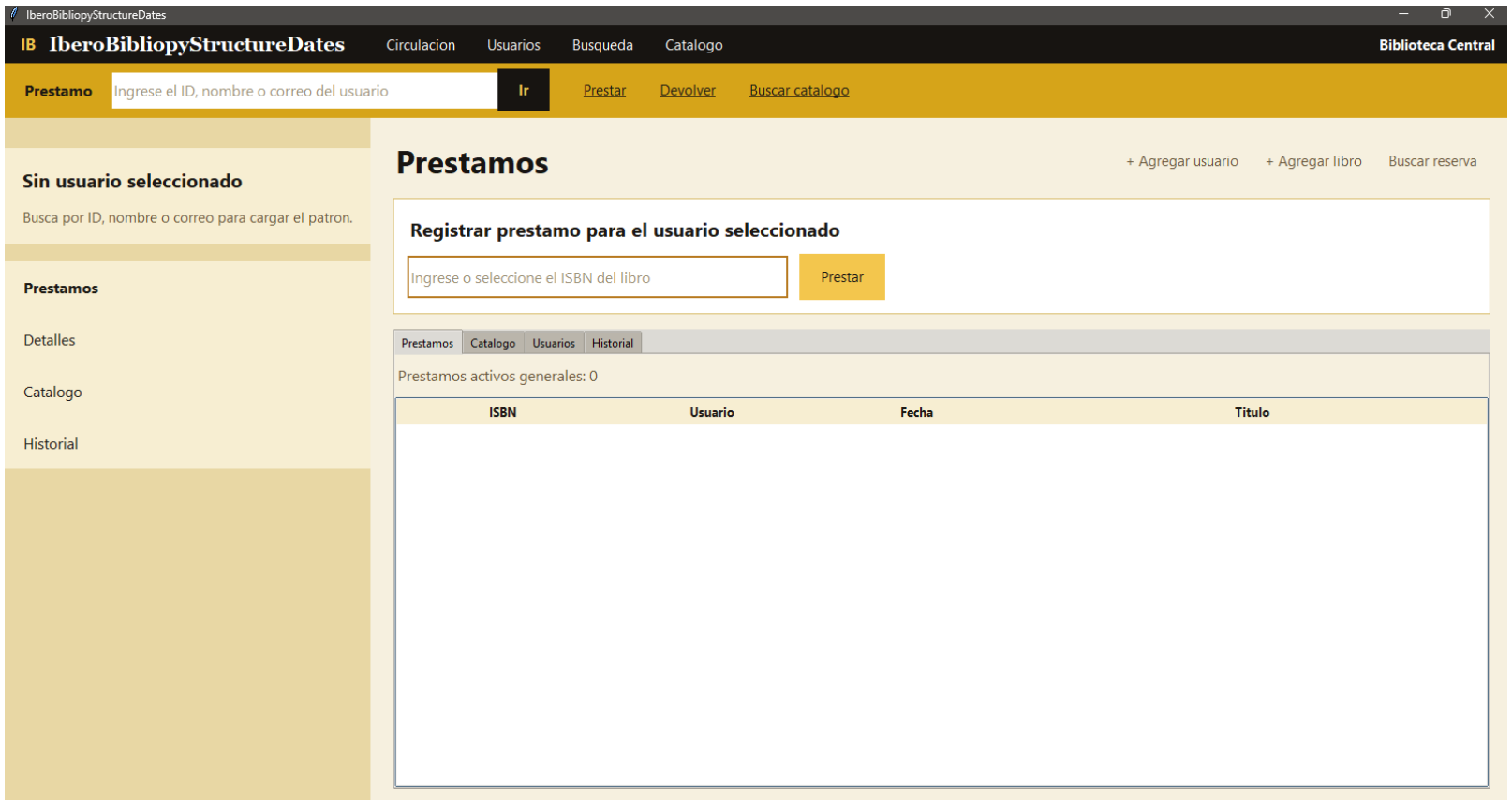


Figura 2. Interfaz principal del sistema IberoBibliopyStructureDates. Fuente: elaboración propia.

El funcionamiento del sistema puede comprenderse de forma secuencial, siguiendo la lógica de uso de una biblioteca real. En primer lugar, el sistema permite el registro de libros. Para ello, el usuario administrador ingresa la información correspondiente a cada ejemplar, como ISBN, título, autor, categoría, cantidad total y cantidad disponible. Una vez confirmados los datos, el libro es añadido a la estructura de almacenamiento principal, permitiendo su consulta posterior dentro del catálogo. Este proceso constituye la base del sistema, ya que sin libros registrados no sería posible ejecutar préstamos, devoluciones ni reservas.

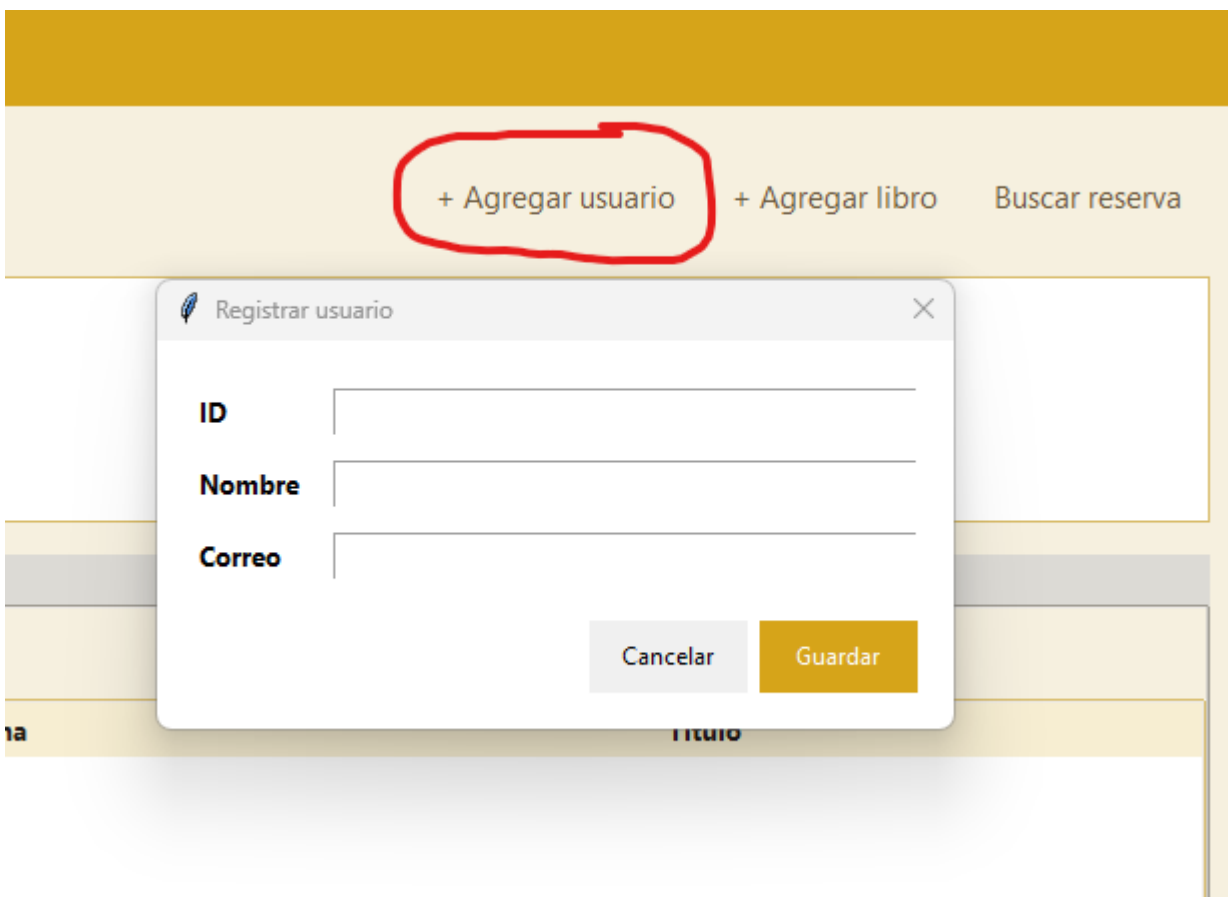


Figura 3. Interfaz de agregar usuarios. Fuente: elaboración propia.

En segundo lugar, el sistema ofrece la opción de registrar usuarios. En esta etapa se almacenan datos como el ID del usuario, su nombre y correo electrónico. La importancia de esta función radica en que todo préstamo debe estar vinculado a una persona específica, de modo que la identificación correcta de los usuarios resulta esencial para mantener consistencia y trazabilidad en la información del sistema.

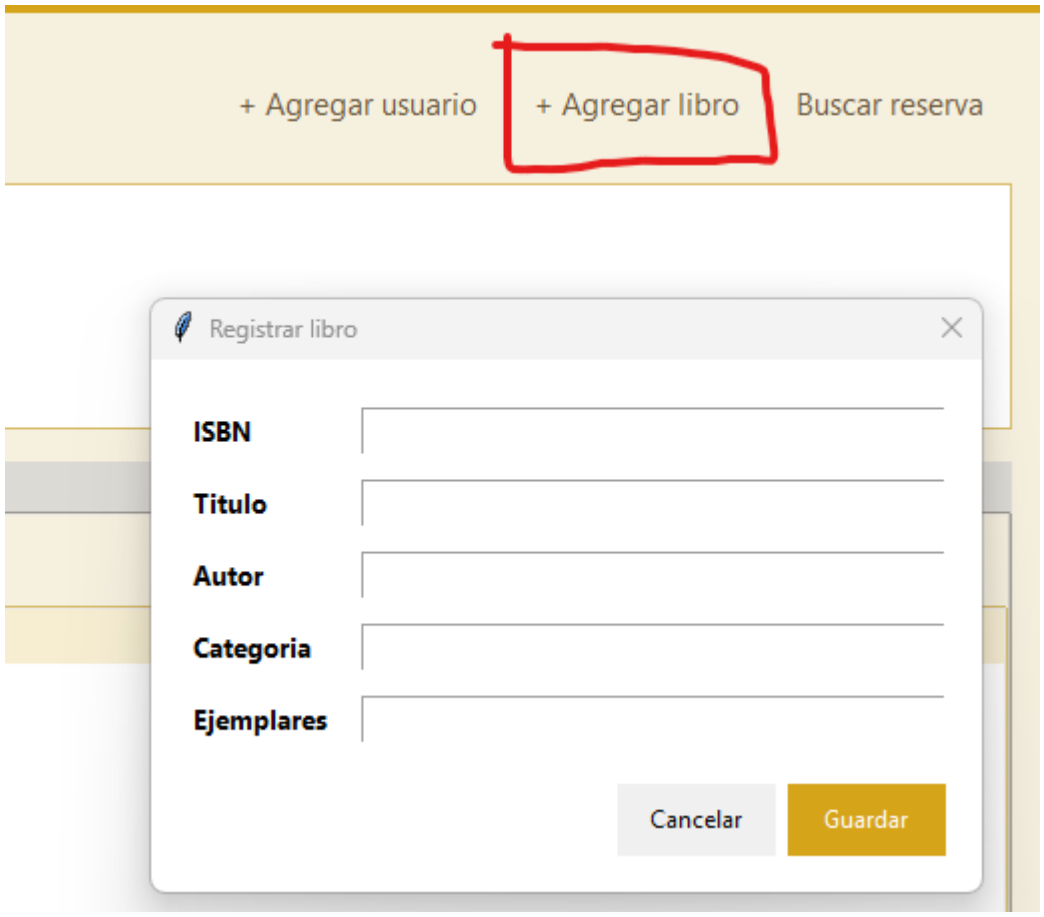


Figura 4. Interfaz de agregar Libros. Fuente: elaboración propia.

Una vez existen libros y usuarios registrados, el prototipo habilita la funcionalidad de búsqueda. En el caso de los libros, el sistema permite localizarlos por ISBN, título o autor. Esta posibilidad mejora la usabilidad del software, ya que no obliga a recordar un único criterio de consulta. En cuanto a los usuarios, la búsqueda puede realizarse por ID, nombre o correo electrónico. Esta flexibilidad facilita el acceso a la información y reduce el tiempo de operación en tareas de circulación.

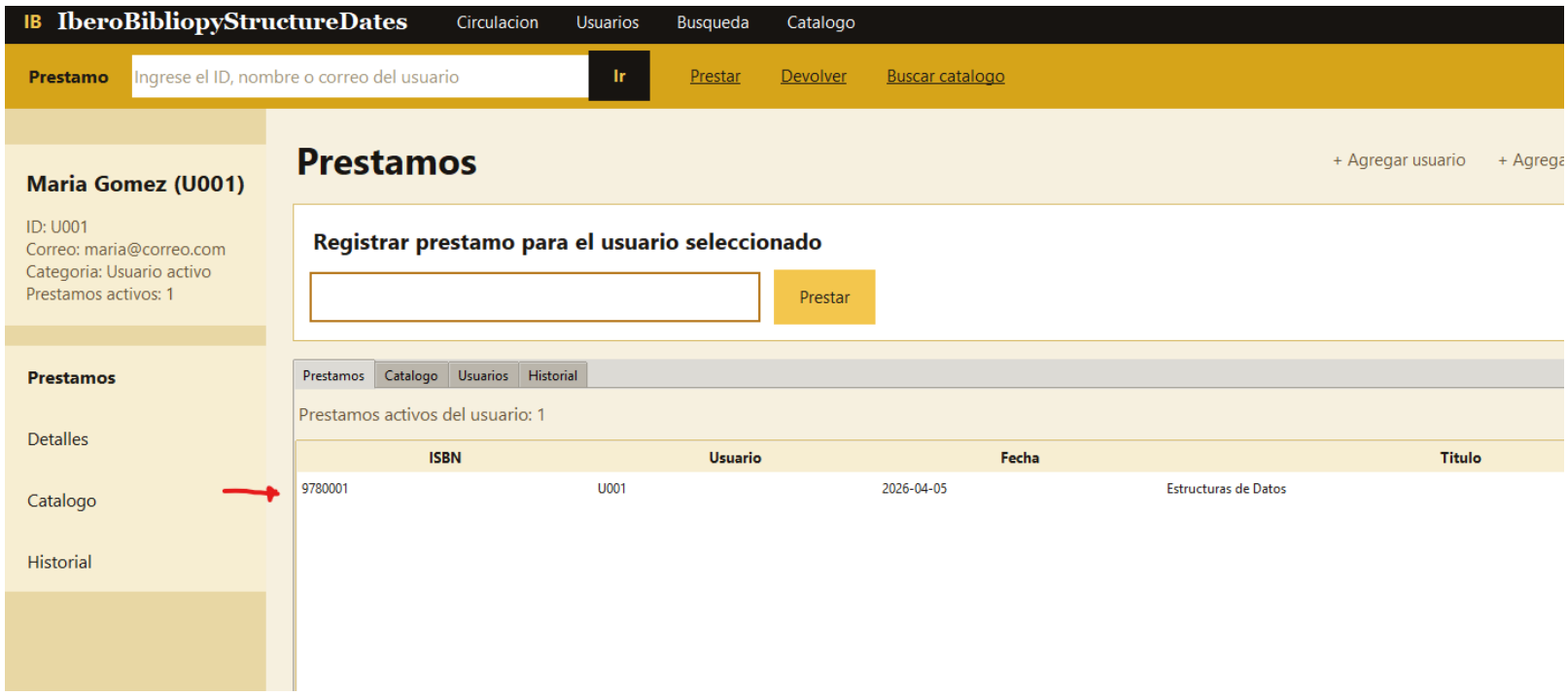


Figura 5. Interfaz de Prestamos. Fuente: elaboración propia.

Figura 6. Interfaz de Catalogo de Libros. Fuente: elaboración propia.

Posteriormente se encuentra el proceso de préstamo de libros, que representa una de las operaciones centrales del sistema. El flujo de funcionamiento es el siguiente: primero se selecciona o identifica al usuario que recibirá el libro; luego se ingresa o selecciona el ISBN del ejemplar; finalmente, el sistema valida si el libro se encuentra disponible. Si hay ejemplares disponibles, se genera el préstamo, se registra la fecha correspondiente y se disminuye la cantidad disponible del libro. Además, el préstamo queda almacenado dentro del listado de préstamos activos y se agrega una entrada al historial de acciones recientes.

Figura 7. Interfaz de Usuarios. Fuente: elaboración propia.

Sin embargo, el sistema también contempla una situación frecuente en la administración bibliotecaria: la falta de disponibilidad de ejemplares. Cuando esto ocurre, el prototipo activa la funcionalidad de reserva mediante cola de espera.

En lugar de rechazar completamente la solicitud, el sistema permite agregar al usuario a la cola de reserva del libro solicitado. Esta decisión es importante porque modela un escenario real y demuestra el uso adecuado de la estructura cola. Cada usuario es incorporado respetando el orden de llegada, lo que garantiza un mecanismo transparente y ordenado de asignación futura.

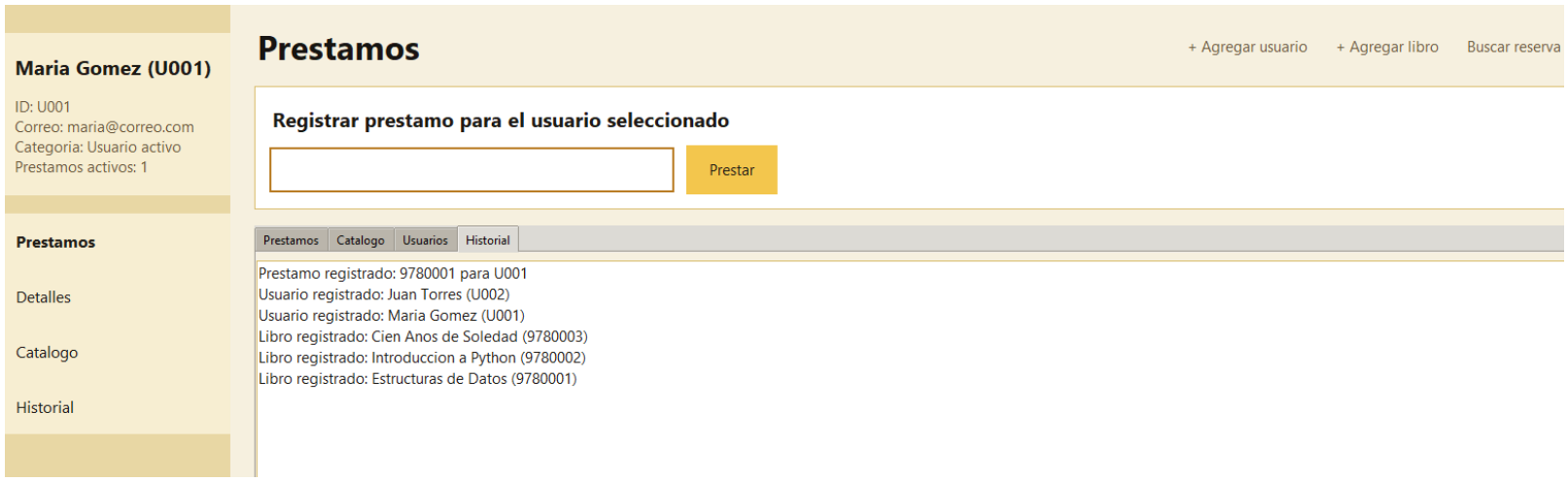


Figura 8. Interfaz de Usuarios. Fuente: elaboración propia.

La operación de devolución también sigue una lógica definida paso a paso. Cuando un usuario devuelve un libro, el sistema actualiza el estado del préstamo correspondiente y aumenta la cantidad disponible del ejemplar. No obstante, antes de dejar el libro simplemente como disponible, el programa verifica si existe una cola de reserva asociada. Si hay usuarios en espera, el sistema asigna automáticamente el libro al primero de la cola, generando un nuevo préstamo y actualizando de nuevo la disponibilidad. Este comportamiento representa uno de los elementos más sólidos del prototipo, ya que articula varias estructuras de datos y muestra cómo la lógica de negocio puede automatizar procesos de administración.

De manera complementaria, el sistema incorpora un módulo de historial, el cual registra las acciones recientes que se han ejecutado. Cada vez que se realiza una operación relevante, como registrar un usuario, añadir un libro, prestar un ejemplar o procesar una devolución, se genera una entrada en esta bitácora. La estructura de pila utilizada para este componente resulta apropiada porque permite consultar primero los eventos más recientes, lo que coincide con la necesidad de seguimiento inmediato dentro del entorno de uso.

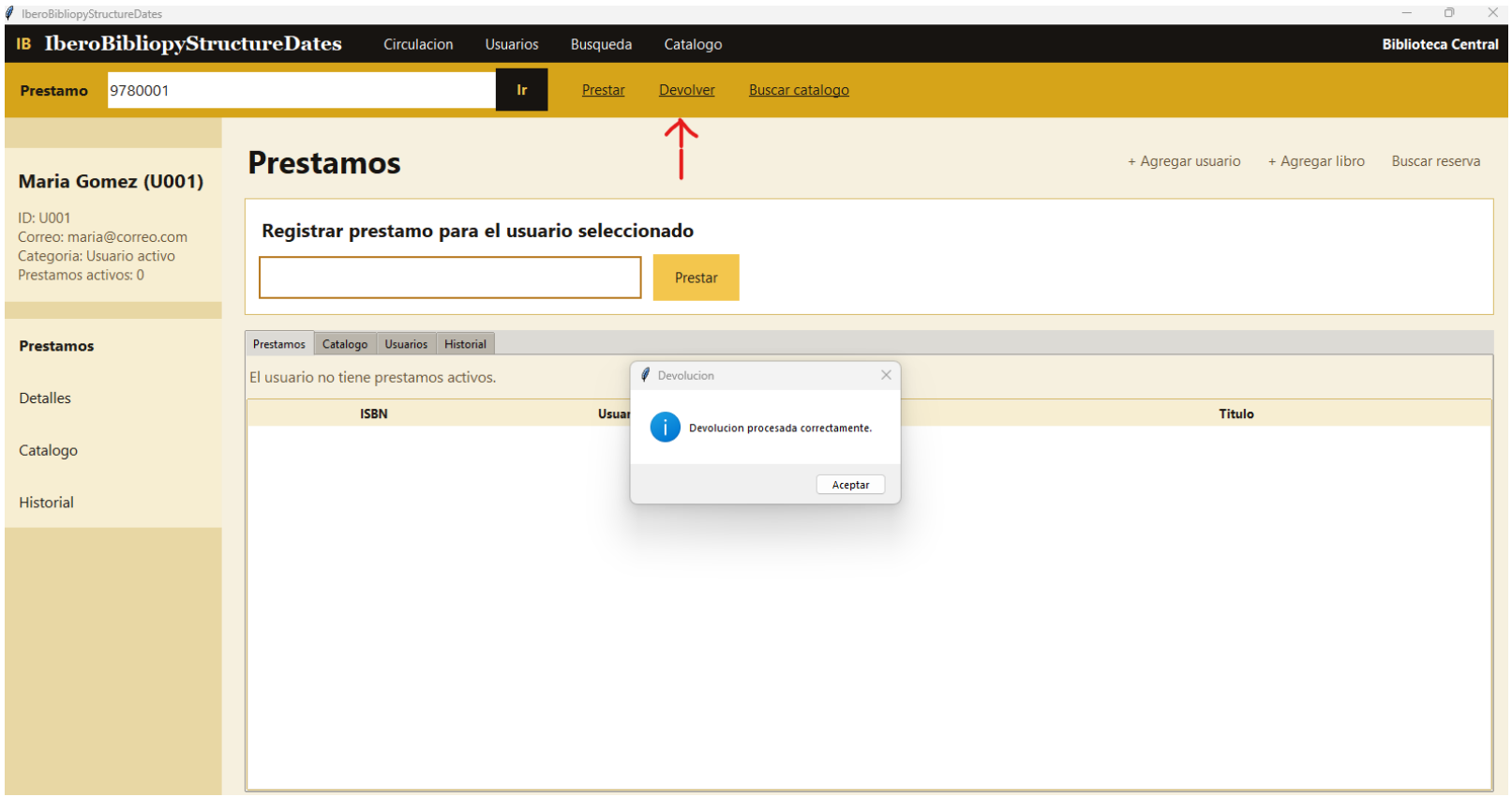


Figura 9. Interfaz de Devolución de Libro. Fuente: elaboración propia.

En lo referente a la presentación visual, la interfaz gráfica organiza sus funciones en áreas claramente diferenciadas. En la parte superior se dispone una barra principal con el nombre del sistema, lo que aporta identidad y contexto al usuario. Junto a ella se encuentra una barra operativa de circulación, desde la cual pueden activarse acciones relacionadas con la gestión de préstamos, devoluciones y búsqueda en el catálogo.

En el costado izquierdo se ubica un panel lateral destinado a mostrar la información del usuario seleccionado. Esta sección es especialmente útil durante el proceso de préstamo, ya que permite confirmar visualmente quién es la persona sobre la cual se está operando. Gracias a ello se reduce el riesgo de asignar libros al usuario equivocado y se mejora la claridad de la interacción.

En la zona central del sistema se encuentra el área principal de trabajo, organizada mediante pestañas. Estas pestañas agrupan la información de préstamos, catálogo, usuarios e historial, lo que favorece una navegación más ordenada. Cada sección presenta sus datos de forma estructurada, permitiendo al usuario consultar el estado actual del sistema sin necesidad de cambiar constantemente de ventana. Además, los formularios emergentes para registrar libros y usuarios facilitan la entrada de información sin recargar visualmente la pantalla principal.

Otro elemento relevante del diseño es la incorporación de campos con texto guía o placeholder, los cuales orientan al usuario sobre la información que debe ingresar. Esta característica, aunque sencilla, mejora notablemente la experiencia de uso, ya que disminuye la ambigüedad y contribuye a una interacción más intuitiva. Del mismo modo, la elección de

una paleta de colores compuesta por negro, amarillo y miel aporta contraste, legibilidad y una estética institucional que se relaciona adecuadamente con el contexto bibliotecario.

En conjunto, el prototipo funcional no se limita a mostrar una interfaz atractiva, sino que integra lógica estructurada, manejo coherente de datos y un flujo operativo consistente. Cada operación fue diseñada para responder a un requerimiento concreto del sistema de biblioteca, y cada estructura de datos se implementó en función de una necesidad identificada previamente. De esta manera, IberoBibliopyStructureDates constituye una propuesta funcional y académicamente pertinente para evidenciar la aplicación de estructuras de datos lineales en un escenario realista.

## 4. Uso y relación de conceptos

El valor académico del prototipo desarrollado se encuentra en la manera en que articula los conceptos estudiados en la asignatura con una aplicación concreta. No se trató únicamente de programar una interfaz o de almacenar datos de forma básica, sino de traducir principios propios de las estructuras de datos lineales en componentes funcionales dentro de un sistema de gestión de biblioteca.

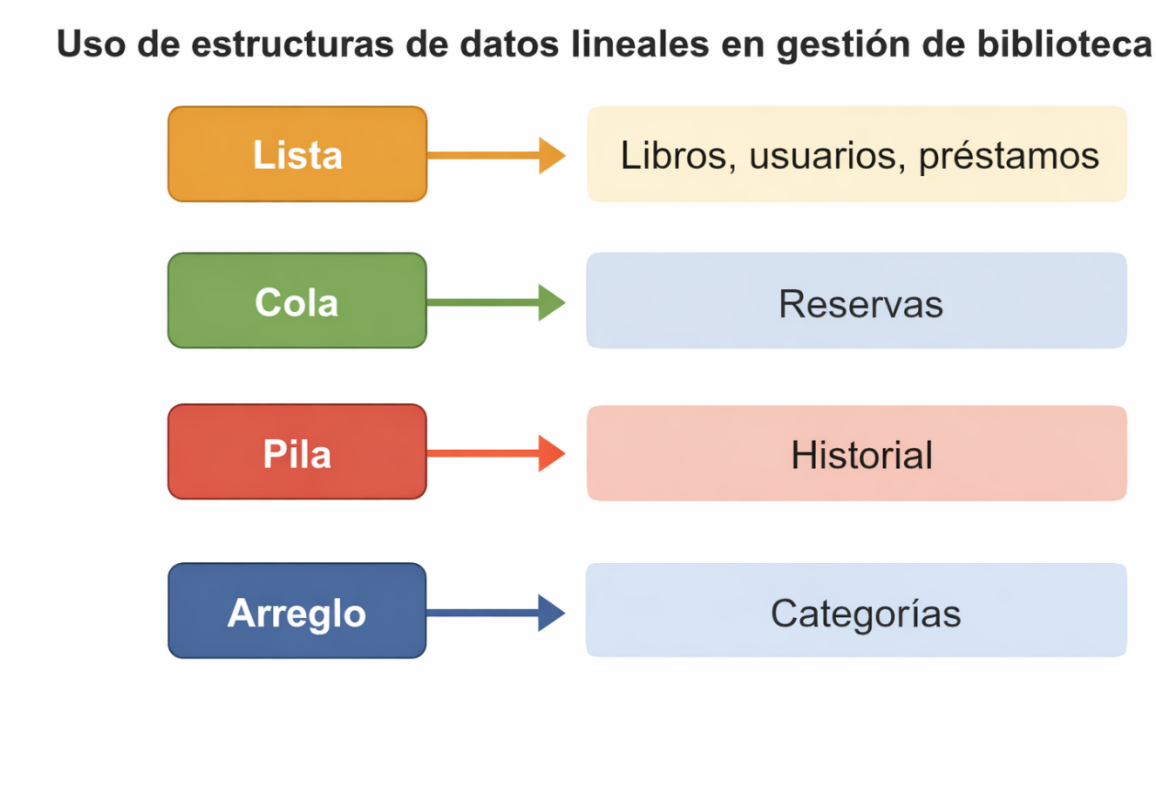


Figura 10. Interfaz de Uso de Estructuras de datos en el proyecto. Fuente: elaboración propia.

Uno de los primeros conceptos aplicados fue el de estructura lineal como mecanismo de organización secuencial de datos. En el prototipo, esta idea se materializa de diferentes maneras según el tipo de información y la lógica de operación esperada. La comprensión de esta diferencia fue determinante para evitar decisiones arbitrarias y construir un sistema coherente desde su base.

La lista fue empleada como estructura principal para almacenar libros, usuarios y préstamos. Su elección se justifica porque estas colecciones crecen de manera dinámica y requieren recorridos frecuentes para ejecutar búsquedas, mostrar registros o validar condiciones. En un sistema bibliotecario, por ejemplo, es común recorrer todos los libros para encontrar coincidencias por autor o título, así como consultar el conjunto de préstamos para identificar cuáles se encuentran activos. La lista, en este sentido, funciona como una estructura flexible, comprensible y suficiente para un prototipo académico de estas características.

La cola, por su parte, fue utilizada para gestionar las reservas de libros sin disponibilidad. Aquí la relación entre concepto y práctica es especialmente clara. La teoría establece que una cola opera bajo el criterio FIFO, donde el primer elemento en entrar es también el primero en salir. Esta lógica coincide exactamente con la dinámica de una lista de espera en una biblioteca: el primer usuario que reserva un libro debe ser el primero en recibirlo cuando el ejemplar sea devuelto. En consecuencia, la selección de esta estructura no solo responde a una conveniencia técnica, sino también a un criterio de justicia operativa y orden administrativo.

La pila fue vinculada al registro del historial de acciones. En este caso, el principio LIFO permite acceder primero al evento más reciente, lo cual tiene sentido cuando se desea revisar las últimas operaciones efectuadas dentro del sistema. Desde una perspectiva conceptual, esta implementación demuestra que la pila no es únicamente una estructura abstracta empleada en ejemplos clásicos, sino una herramienta válida para organizar eventos temporales dentro de un entorno de software real.

El arreglo de tamaño fijo se utilizó para representar las categorías base de la biblioteca. Esta decisión permite ilustrar otro principio importante: no toda estructura debe elegirse por su capacidad de crecimiento. Cuando se sabe de antemano que un conjunto de datos será relativamente estable, un arreglo fijo resulta una opción razonable, ya que expresa orden, previsibilidad y control sobre la organización de la información. En este sentido, el proyecto también permitió comprender que la pertinencia de una estructura depende del comportamiento esperado de los datos y no solo de su popularidad o facilidad de uso.

Otro concepto central fue el de parametrización de datos. Este principio orientó la definición de atributos para cada entidad del sistema. En lugar de almacenar información de manera improvisada, se establecieron parámetros concretos para libros, usuarios y préstamos, procurando que cada uno respondiera a una necesidad funcional. Así, el ISBN se definió como identificador del libro; el ID permitió distinguir a cada usuario; y la fecha del préstamo, junto con el estado de devolución, facilitó el seguimiento de la circulación de ejemplares. Esta organización evidencia que una correcta selección de parámetros mejora la consistencia lógica y facilita la implementación de

operaciones posteriores.

También se hizo evidente la relación entre abstracción de datos y resolución de problemas reales. Una biblioteca posee múltiples procesos: registrar material, consultar disponibilidad, asignar préstamos, gestionar devoluciones y atender reservas. Cada uno de estos procesos puede entenderse como un problema de administración de información. Las estructuras de datos lineales permiten abstraer estos problemas y ofrecerles una solución ordenada. De esta manera, el proyecto confirma que la teoría de estructuras de datos no se limita a definiciones formales, sino que constituye una base para construir sistemas funcionales y comprensibles.

Además, el desarrollo del prototipo involucró saberes procedimentales relacionados con el diseño e implementación de software. Fue necesario analizar requisitos, seleccionar estructuras, definir atributos, construir clases o módulos, enlazar la lógica con la interfaz y validar el comportamiento del sistema. Esta secuencia permitió poner en práctica competencias de análisis, modelado y programación, así como fortalecer la comunicación técnica de las decisiones tomadas.

En el plano actitudinal, el proyecto también evidenció la importancia de la precisión, el orden y la responsabilidad en el manejo de la información. Aunque el prototipo se desarrolló con fines académicos, trabaja con datos asociados a usuarios de biblioteca, lo que hace pertinente reflexionar sobre el tratamiento adecuado de la información personal. Asimismo, la construcción de una interfaz clara y la organización lógica del sistema muestran un compromiso con la calidad del diseño y con la experiencia del usuario.

Por tanto, la relación entre teoría y práctica en este proyecto no es superficial. Cada estructura implementada responde a un concepto estudiado, cada operación funcional se vincula con una necesidad real del sistema y cada decisión de diseño busca mantener coherencia con el propósito formativo del curso. En conjunto, el prototipo IberoBibliopyStructureDates constituye una evidencia concreta de cómo los conceptos de estructuras de datos lineales pueden ser comprendidos, justificados y aplicados de forma efectiva en el desarrollo de software.

## 5. Pruebas y Validación

La etapa de pruebas y validación fue fundamental para verificar que el prototipo IberoBibliopyStructureDates no solo cumpliera una función visual, sino que operara de manera coherente con la lógica definida desde el diseño. En un sistema de gestión de biblioteca, cada operación depende de la consistencia de los datos almacenados y de la correcta interacción entre las estructuras utilizadas. Por ello, las pruebas realizadas tuvieron como objetivo comprobar tanto la funcionalidad individual de cada módulo como la integración general del sistema.

La primera validación se centró en el registro de libros. En esta prueba se verificó que el sistema aceptara

correctamente los datos de entrada, que almacenara cada libro dentro de la estructura correspondiente y que la información quedara disponible para consultas posteriores. Esta validación fue importante porque el libro constituye una de las entidades centrales del sistema. Si su registro presentara errores, se verían comprometidas operaciones posteriores como la búsqueda, el préstamo o la devolución.

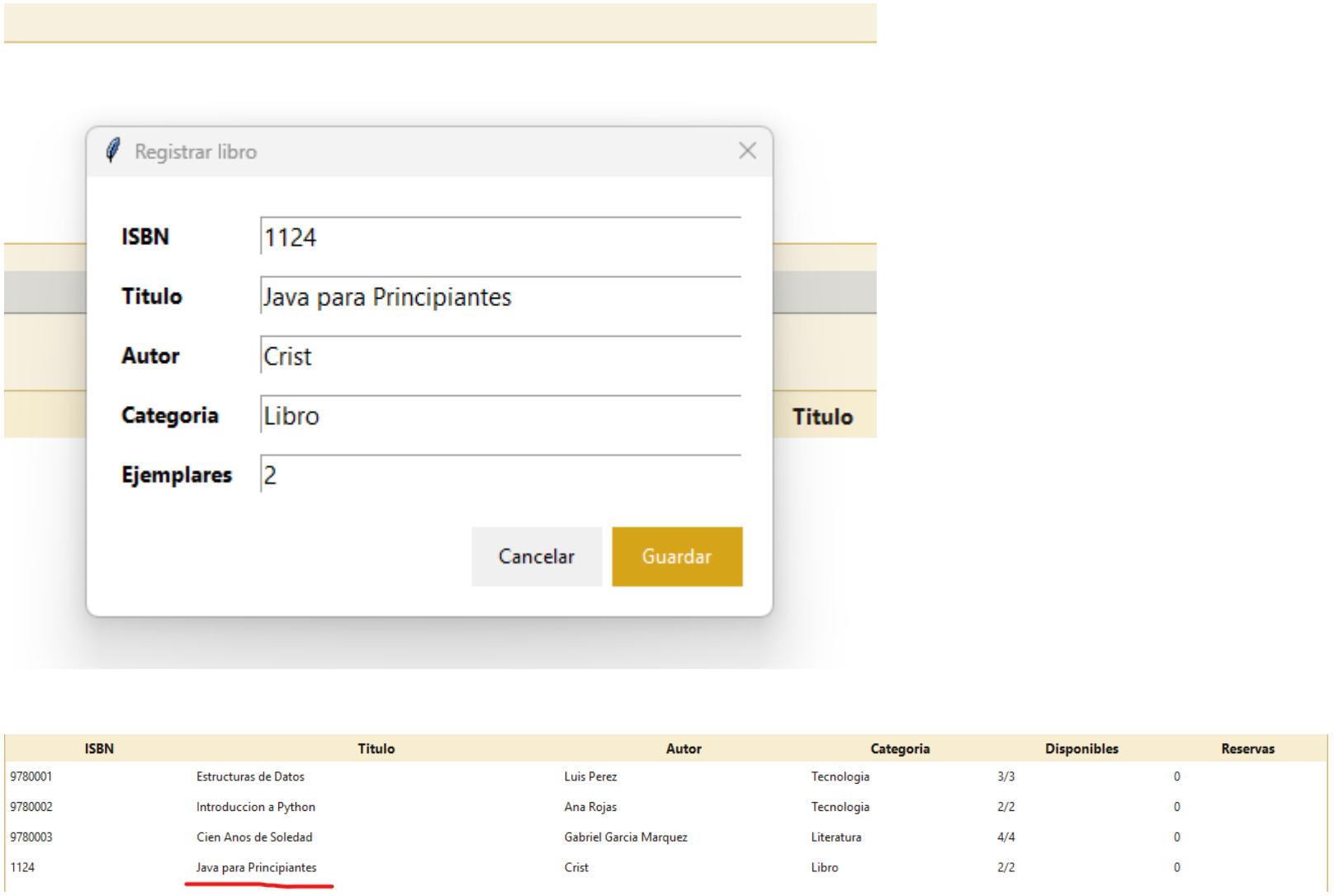


Figura 11. Interfaz y Validación de Registro de Libro. Fuente: elaboración propia.

En segundo lugar, se probaron los mecanismos de registro de usuarios. Se verificó que el sistema incorporara adecuadamente los datos básicos de identificación, que permitiera su búsqueda posterior y que el usuario pudiera ser seleccionado para asociarlo con operaciones de circulación. Esta prueba fue relevante porque la lógica del préstamo depende directamente de la existencia de usuarios válidos dentro del sistema.

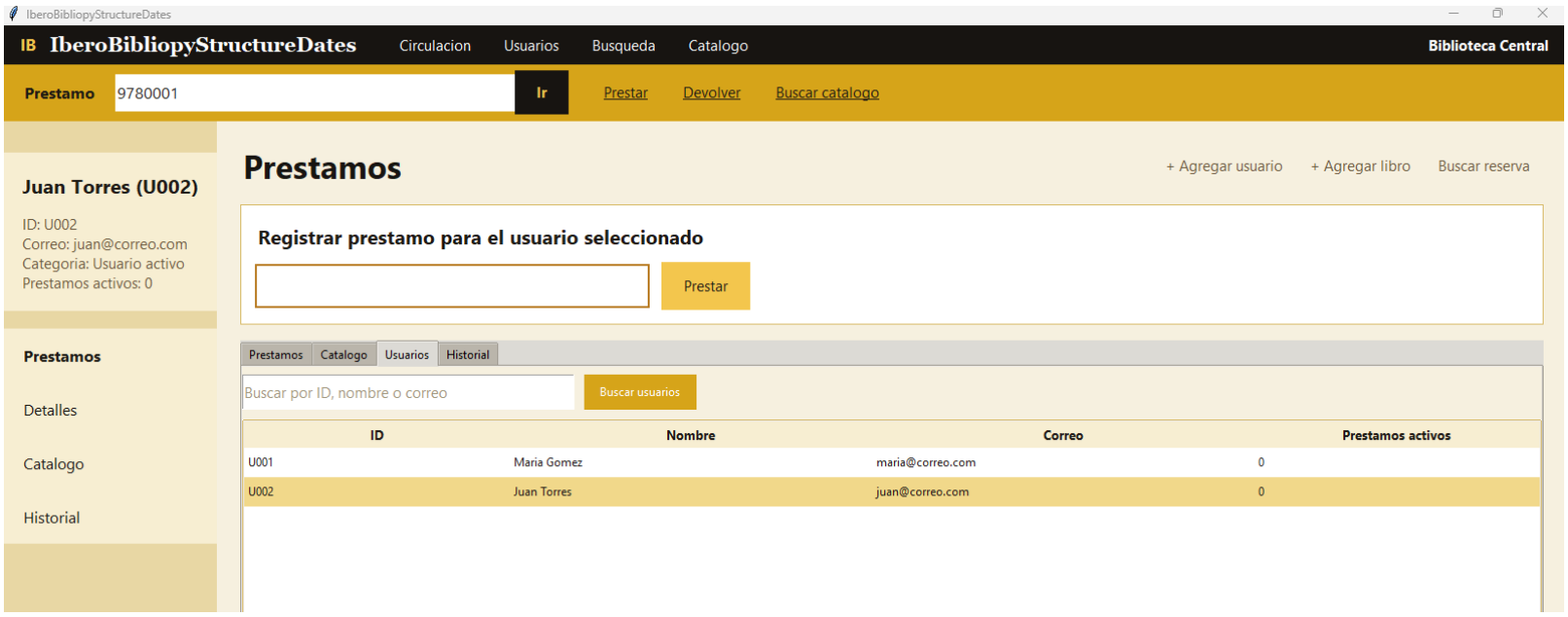


Figura 12. Interfaz y Validación de Selección de Usuarios. Fuente: elaboración propia.

También se realizaron pruebas de búsqueda de libros, comprobando que el prototipo respondiera correctamente a consultas por ISBN, título y autor. La importancia de esta validación radica en que la búsqueda es una de las funciones de uso más frecuente en un entorno bibliotecario. Una falla en este proceso afectaría tanto la consulta del catálogo como la localización rápida de ejemplares para préstamo o reserva.

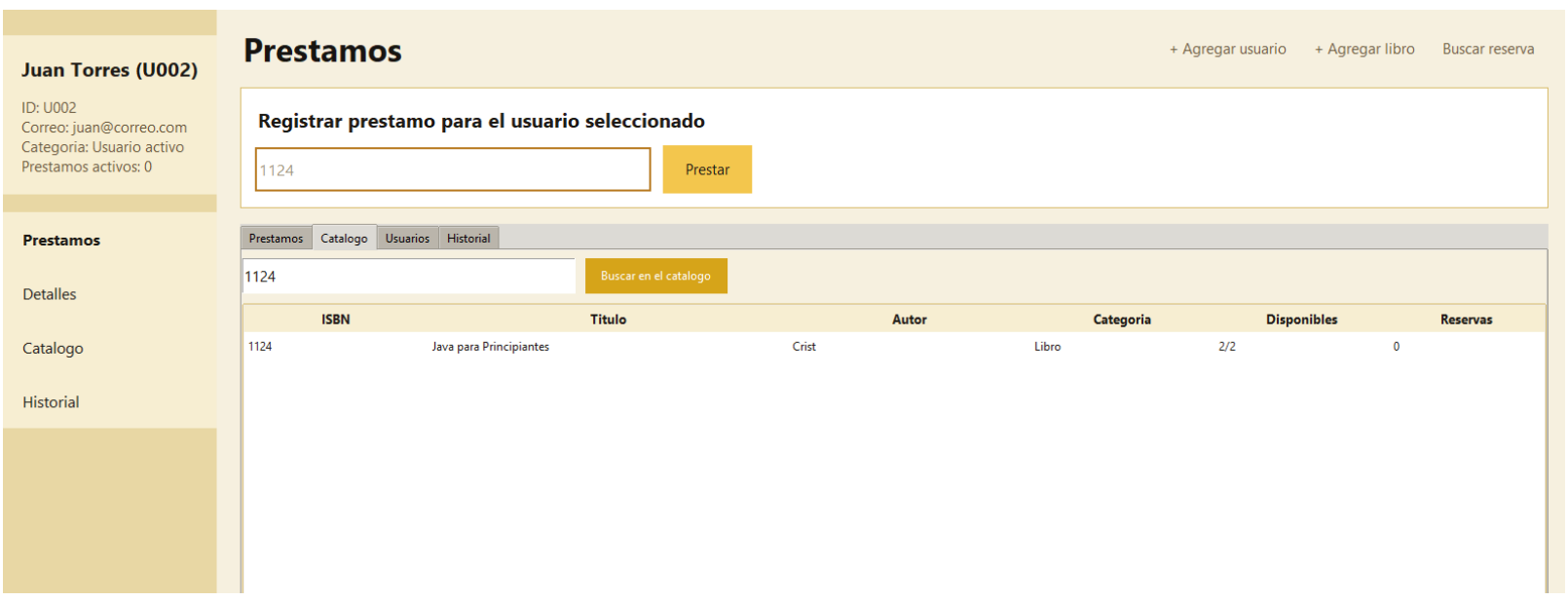


Figura 13. Interfaz y Validación de Búsqueda de Libros por ISBN. Fuente: elaboración propia.

A continuación, se validó el proceso de préstamo con disponibilidad. Para ello, se seleccionó un usuario, se ingresó un libro con ejemplares disponibles y se verificó que el sistema registrara el préstamo, disminuyera la cantidad disponible y reflejara la operación en la lista de préstamos activos y en el historial. Esta prueba confirmó que la lógica principal de circulación funcionaba correctamente y que la actualización de estados era consistente con la acción ejecutada.

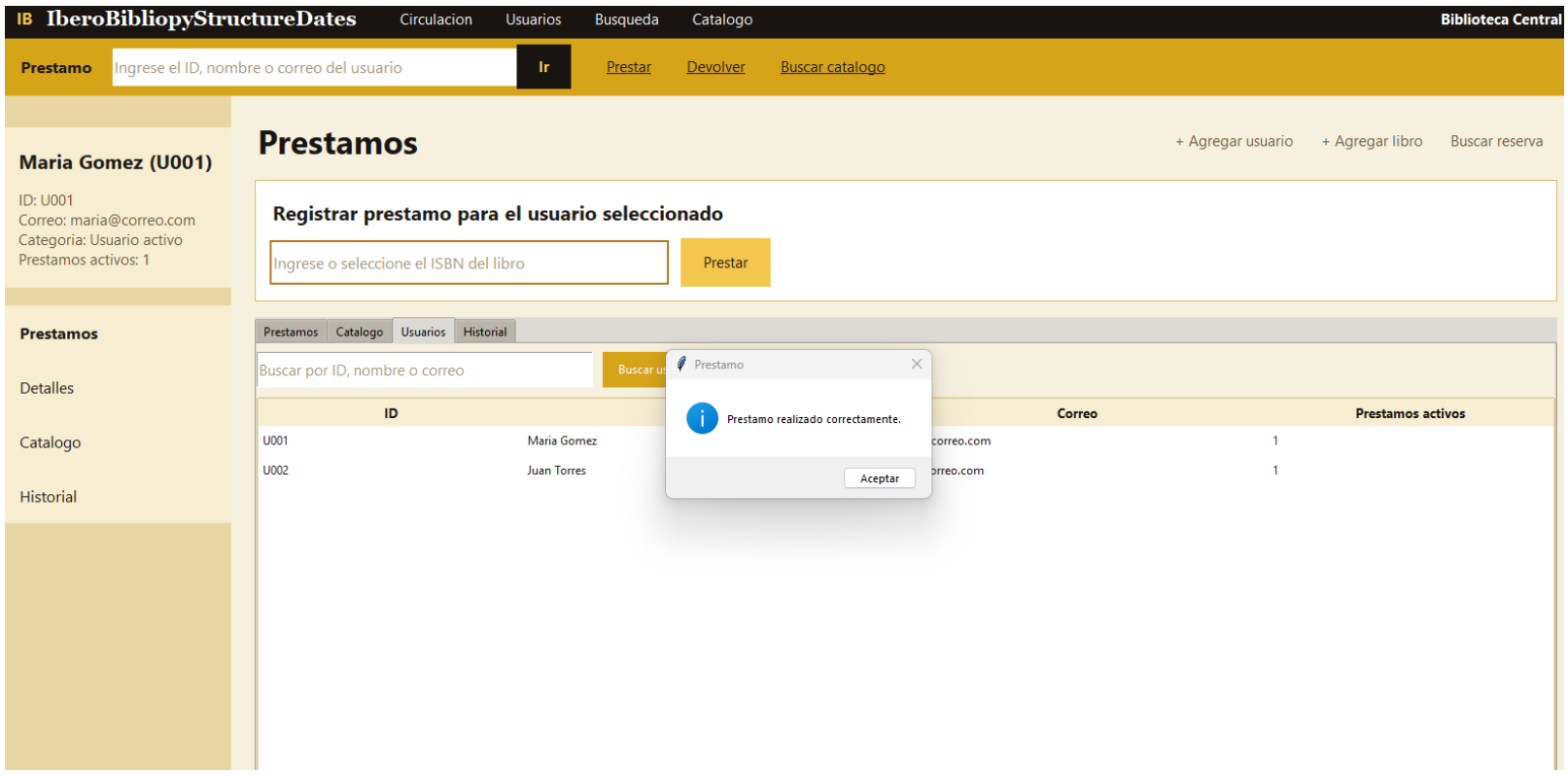


Figura 14. Interfaz y Validación de Disponibilidad de libros. Fuente: elaboración propia.

Una de las validaciones más importantes correspondió al escenario de reserva cuando no hay ejemplares disponibles. En este caso, se comprobó que, ante la ausencia de disponibilidad, el sistema no generara un préstamo incorrecto, sino que incorporara al usuario a la cola de espera del libro. Esta prueba fue esencial porque permitió comprobar el uso adecuado de la estructura cola y verificar que el orden de atención quedara efectivamente registrado.

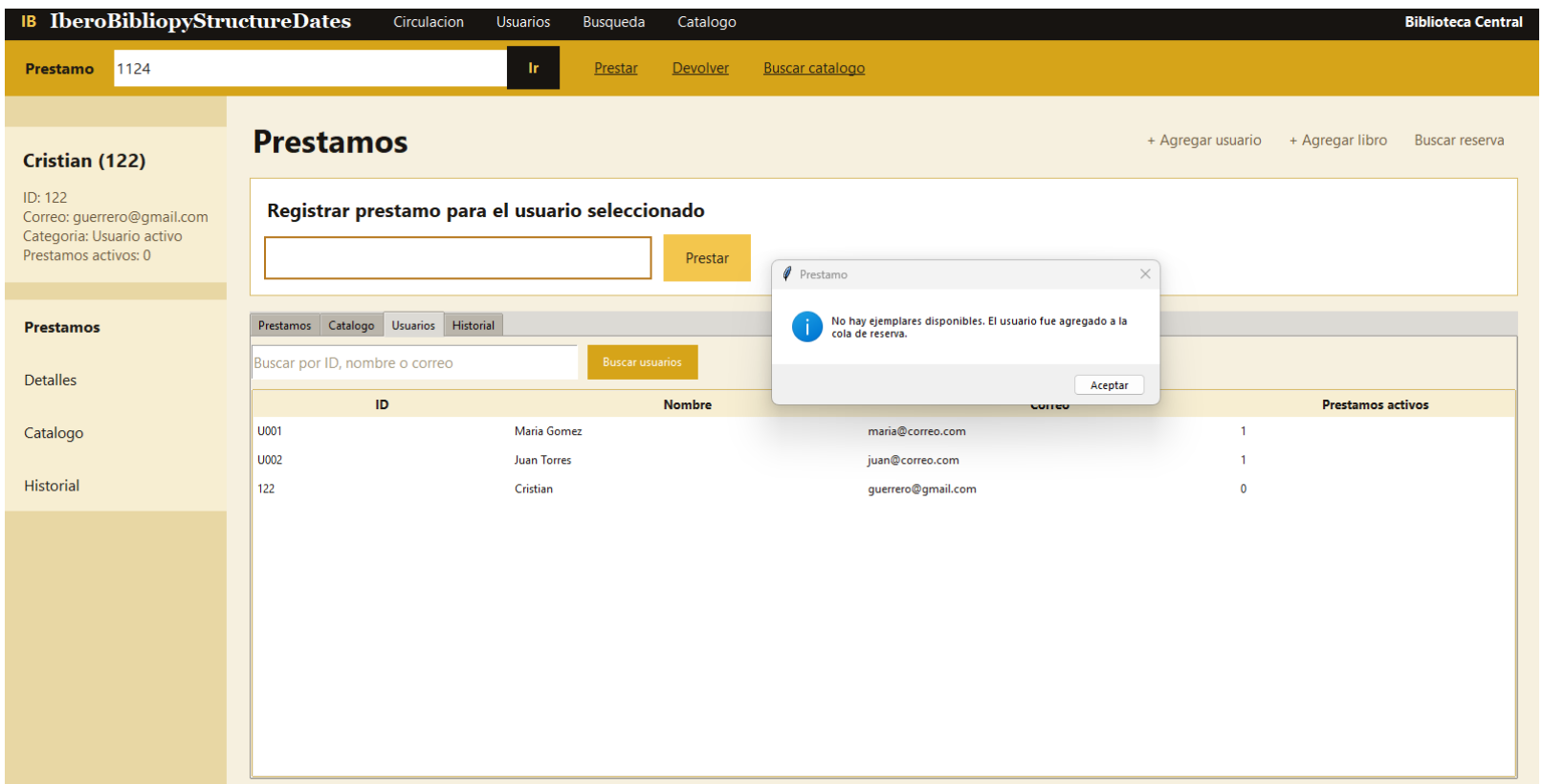


Figura 15. Interfaz y Validación de Disponibilidad de libros. Fuente: elaboración propia.

De manera complementaria, se probó la operación de devolución con asignación automática al siguiente usuario en cola. Esta validación tuvo especial relevancia, ya que involucró la interacción entre varias estructuras de datos y varios

estados del sistema. Se comprobó que, al devolver un libro, el programa actualizara el préstamo correspondiente, incrementara inicialmente la disponibilidad y luego verificara si existían reservas pendientes. Cuando la cola contenía usuarios en espera, el sistema debía asignar automáticamente el libro al primero de la fila. Esta prueba permitió confirmar que la lógica de automatización se ejecutaba correctamente y que la estructura FIFO era respetada.

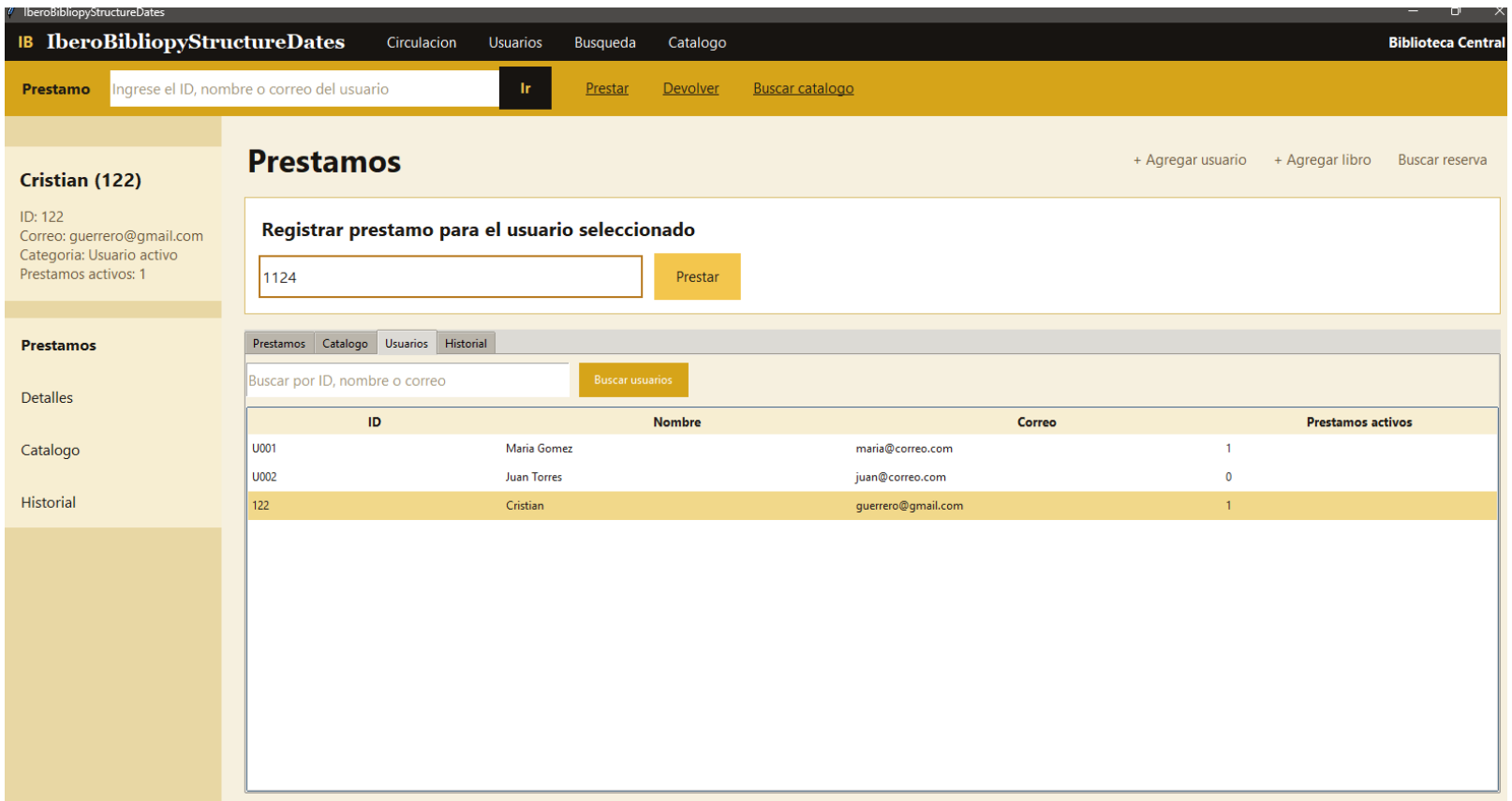


Figura 16. Validación Asignación de libros Automática. Fuente: elaboración propia.

Adicionalmente, se realizó una validación general mediante pruebas unitarias en Python. Estas pruebas tuvieron como finalidad comprobar segmentos específicos de la lógica interna del sistema, aislando operaciones concretas para verificar su correcto funcionamiento. Las pruebas unitarias son especialmente importantes porque permiten identificar errores de forma temprana, asegurar la estabilidad del código y facilitar futuras mejoras o correcciones. Gracias a ellas, fue posible reforzar la confiabilidad del prototipo más allá de la revisión visual de la interfaz.

Figura 17. Validación Pruebas Unitarias Lógica Interna. Fuente: elaboración propia.

Desde una perspectiva metodológica, la etapa de pruebas no solo sirvió para detectar errores, sino también para confirmar que las decisiones de diseño habían sido apropiadas. Cuando una estructura de datos está bien elegida, las operaciones relacionadas con ella tienden a comportarse de forma natural y coherente. En este proyecto, la correcta respuesta de los módulos de reserva, historial y préstamos fue un indicador de que la selección de lista, cola, pila y arreglo respondió adecuadamente a los requisitos establecidos.

Las pruebas también permitieron evaluar la usabilidad de la interfaz. Se observó que la organización en paneles y pestañas facilitaba la navegación, que los textos guía reducían la ambigüedad al diligenciar formularios y que la distribución visual de la información ayudaba a identificar rápidamente el estado del sistema. En otras palabras, la validación no se limitó a la lógica de programación, sino que también consideró la experiencia de interacción del usuario con el prototipo.

En conjunto, la fase de pruebas y validación permitió concluir que el sistema desarrollado cumple con los requerimientos esenciales del proyecto. Las funciones principales fueron ejecutadas de manera satisfactoria, la lógica de las estructuras de datos se comportó según lo esperado y la interfaz ofreció condiciones adecuadas de operación. Por esta razón, puede afirmarse que el prototipo no solo representa una propuesta académica bien fundamentada, sino también una solución funcional y coherente con los objetivos del curso.

## 6. Conclusiones

El desarrollo del prototipo IberoBibliopyStructureDates permitió evidenciar que las estructuras de datos lineales constituyen herramientas fundamentales para la construcción de soluciones informáticas organizadas, funcionales y coherentes con problemas reales. A través del diseño de un sistema de gestión de biblioteca, fue posible comprobar que conceptos como listas, colas, pilas y arreglos adquieren verdadero sentido cuando se aplican a procesos concretos de almacenamiento, administración y recuperación de información.

Una de las principales conclusiones del proyecto es que la correcta selección de estructuras de datos influye directamente en la calidad del sistema. No todas las necesidades de almacenamiento o procesamiento pueden resolverse con una única estructura. En este trabajo, la lista resultó pertinente para manejar colecciones dinámicas de libros, usuarios y préstamos; la cola fue adecuada para respetar el orden de espera en las reservas; la pila facilitó el registro de acciones recientes; y el arreglo fijo permitió representar categorías base de forma estable. Esta diferenciación demostró que la relación entre estructura y función debe ser analizada cuidadosamente desde el inicio del diseño.

Asimismo, el proyecto permitió fortalecer la comprensión del concepto de parametrización de datos. Definir con claridad qué información debía almacenarse para cada entidad del sistema fue una condición indispensable para lograr consistencia interna, facilitar las búsquedas y garantizar que las operaciones de préstamo, devolución y reserva funcionaran correctamente. En este sentido, el trabajo confirmó que un sistema bien diseñado comienza por una adecuada organización lógica de sus datos.

Otra conclusión relevante es que la programación de un sistema funcional no depende únicamente de la lógica algorítmica, sino también de la manera en que dicha lógica se comunica al usuario. La interfaz desarrollada en Tkinter aportó claridad, orden y facilidad de interacción, permitiendo que las funciones del sistema fueran comprensibles y accesibles. Esto demuestra que la experiencia de usuario también forma parte de la calidad del software, incluso en proyectos académicos.

De igual forma, la etapa de pruebas permitió validar que el prototipo responde adecuadamente a los requerimientos planteados. La verificación de registros, búsquedas, préstamos, reservas y devoluciones automáticas mostró que la lógica implementada es consistente y que las estructuras de datos elegidas se comportan conforme a lo esperado. La inclusión de pruebas unitarias reforzó además la confiabilidad del sistema y evidenció una preocupación por la calidad del desarrollo.

En el plano formativo, este proyecto contribuyó al fortalecimiento de competencias tanto técnicas como analíticas. No solo implicó programar, sino también interpretar requisitos, justificar decisiones, estructurar información, evaluar comportamientos y comunicar resultados de manera académica. Por ello, la experiencia resultó valiosa para comprender que el estudio de estructuras de datos no es un ejercicio aislado, sino una base indispensable para el diseño de software sólido y bien fundamentado.

En conclusión, IberoBibliopyStructureDates cumple con el objetivo de demostrar el uso correcto de estructuras de datos lineales en un sistema de gestión de biblioteca desarrollado en Python. El prototipo integra teoría y práctica, responde a necesidades funcionales reales y evidencia una aplicación consciente de los conocimientos abordados en la asignatura. Más allá de su valor como entregable académico, el proyecto deja como aprendizaje central que toda solución informática eficaz comienza con una adecuada comprensión de los datos, sus relaciones y la estructura más conveniente para administrarlos.

## 7. Referencias

Ayala San Martín, G. (2020). *Algoritmos y programación: mejores prácticas*. Fundación Universidad de las Américas Puebla (UDLAP).

Fritelli, V., Guzmán, A., y Tymoschuk, J. (2020). *Algoritmos y estructuras de datos* (2.ª ed.). Jorge Sarmiento Editor - Universitas.

Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*. El Cid Editor.

Zohonero Martínez, I., y Joyanes Aguilar, L. (2008). *Estructuras de datos en Java*. McGraw-Hill.